

The GGUF (GGML Universal File) file format is a binary file format that stores both tensors and metadata in a single file and is designed for fast saving and loading of model data. It was introduced in August 2023 by the llama.cpp project to better maintain backward compatibility as support was added for other model architectures. It superseded previous formats used by the project such as GGML and is typically produced by converting models developed with a different machine-learning library such as PyTorch.

GGUF has become the standard format for distributing quantized large language models for local inference and is natively supported by tools such as llama.cpp, Ollama, LM Studio, GPT4All, Jan, and koboldcpp. As of 2026, tens of thousands of GGUF checkpoints are hosted on Hugging Face, which provides first-class integration, including a metadata viewer, an inference endpoint service, and a JavaScript parser library.

History

The model file formats used by the llama.cpp project evolved through four named stages: GGML, GGUF, GGJT, and GGUF. The original GGML format was a thin tensor container that hard-coded model hyperparameters and tokenizer information inside the loader; adding support for a new model architecture or quantization scheme typically required code changes that broke compatibility with existing files.

As llama.cpp grew during 2023 to support additional architectures beyond Llama — including Mistral, Falcon, and others — these limitations became increasingly difficult to manage. GGUF was introduced on 21 August 2023 as a backward-incompatible successor, with the specification formalized through pull request #302 in the ggerganov/ggml repository. The format inherits GGJT's overall layout but replaces its flat hyperparameter list with a structured key-value metadata system, allowing new fields — architecture details, tokenizer vocabularies, training parameters — to be added without modifying the loader or breaking older models.

The format itself has gone through three internal versions. Version 1 established the basic

structure; version 2 added explicit alignment padding to support memory mapping; and version 3, the current version, added optional big-endian support.

Design

GGUF focuses on quantization, the act of reducing precision in the model weights. This can lead to reduced memory usage and increased speed, albeit at the cost of reduced model accuracy.

The format is designed to be:

Self-contained — a single file holds the tensors, the tokenizer and all metadata needed to load and run the model, eliminating the need for accompanying configuration files.

Memory-mappable — tensor data are aligned (by default to a 32-byte boundary) so that weights can be accessed directly through pointers without loading the entire file into RAM, allowing models larger than available memory to be served through operating-system paging.

Extensible — the key-value metadata block lets new fields be added without breaking compatibility with older readers.

GGUF supports 2-bit to 8-bit quantized integer types, common floating-point data formats such as float32, float16 and bfloat16, and 1.58-bit quantization. Several "K-quant" variants (such as Q4_K, Q5_K and Q6_K) use a block-based scheme with separate scale and minimum values per super-block, generally giving better quality at a given bit-width than the simpler legacy quantizations such as Q4_0 and Q8_0.

GGUF contains the information necessary for running a GPT-like language model such as the tokenizer vocabulary, context length, tensor info and other attributes.

File structure

A GGUF file consists of four sequential sections: a fixed-size header, a key-value metadata

block, a tensor information block, and the tensor data itself.

Byte-level structure (little-endian)

Prior to version 3, files were implicitly little-endian; version 3 permits big-endian storage but does not include a flag indicating endianness, so it must be inferred from context.

Metadata block

The metadata block is a sequence of typed key-value pairs. Keys are namespaced strings (for example `general.*`, `tokenizer.*`, or an architecture-specific prefix such as `llama.*`), and values may be scalars, strings, or arrays — including multi-dimensional arrays.

Tensors info block

For each tensor, the info block stores its name, number of dimensions, shape, data type and byte offset within the subsequent `tensor_data[]` region. The naming scheme is standardized across architectures (for example, `blk.0.ffn_gate.weight`), so that loaders can locate weights regardless of the source framework.

Tensor data

Tensor data follows the info block and begins at the next alignment boundary. The alignment value is itself stored in the metadata under the key `general.alignment`; if absent, it defaults to 32 bytes. Aligning the data this way is what allows the file to be memory-mapped directly: tensor weights can be read through pointers without an additional copy, which is important for SIMD operations, GPU DMA transfers and CPU cache efficiency.

Tooling

Models stored in other frameworks are typically converted to GGUF using the

`convert_hf_to_gguf.py` script bundled with `llama.cpp`, which reads Hugging Face checkpoints (commonly in safetensors form) and emits a GGUF file in a chosen base precision such as f16 or bf16. The resulting file can then be re-quantized to one of the GGUF integer formats with the `llama-quantize` utility, and very large models can be sharded across multiple files with `llama-gguf-split`.

The `llama.cpp` project also exposes a C/C++ API (declared in `ggml/include/gguf.h`) and a Python package, `gguf-py`, for reading and writing GGUF files programmatically.

Adoption

GGUF is the native model format of `llama.cpp` and of Ollama, which uses `llama.cpp` as its inference backend; Ollama models pulled from its registry are GGUF files internally, and arbitrary GGUF files from Hugging Face can be loaded by referencing them as `hf.co/{user}/{repo}`. Other inference applications that consume GGUF directly include LM Studio, GPT4All, Jan and koboldcpp.

Hugging Face supports the format as a first-class citizen on its model hub, providing a GGUF metadata viewer, filtering by the `gguf` tag, an inference endpoints integration, and a large collection of community-uploaded GGUF checkpoints.

See also

`llama.cpp`

Open Neural Network Exchange (ONNX)

Large language model

References

External links

Official GGUF specification on GitHub

